Learning to Localize Reference Trajectories in Image-Space for Visual Navigation

Finn Lukas Busch, Matti Vahs, Quantao Yang, Jesús Gerardo Ortega Peimbert,
Yixi Cai, Jana Tumova, Olov Andersson

***Abstract*—We present LoTIS, a model for visual navigation that provides robot-agnostic image-space guidance by localizing a reference RGB trajectory in the robot's current view, without requiring camera calibration, poses, or robot-specific training. Instead of predicting actions tied to specific robots, we predict the image-space coordinates of the reference trajectory as they would appear in the robot's current view. This creates robot-agnostic visual guidance that easily integrates with local planning. Consequently, our model's predictions provide guidance zero-shot across diverse embodiments. By decoupling perception from action and learning to localize trajectory points rather than imitate behavioral priors, we enable a cross-trajectory training strategy for robustness to viewpoint and camera changes. We outperform state-of-the-art methods by 20-50 percentage points in success rate on conventional forward navigation, achieving 94-98% success rate across diverse sim and real environments. Furthermore, we achieve over 5× improvements on challenging tasks where baselines fail, such as backward traversal. The system is straightforward to use: we show how even a video from a phone camera directly enables different robots to navigate to any point on the trajectory. Videos, demo, code and an extended version of the paper are available at https://finnbusch.com/lotis.**

I. Introduction

Visual navigation enables robots to navigate environments using only camera observations. Early work focused on navigating to a single goal image. To enable long range navigation, subsequent work has considered visual reference trajectories, i.e. sequences of unposed RGB images recorded along a path that capture the full route to be followed. Such trajectories can be recorded with any camera, from a handheld smartphone to a robot-mounted sensor, allowing routes to be defined through demonstration without requiring metric maps.

State-of-the-art approaches typically address this task via end-to-end learning, training policies that map current observations and a goal image directly to robot actions [11, 13, 15]. To follow trajectories, these methods extract a single subgoal image from the trajectory and use this as the goal image for the learned policy. We identify three limitations. First, outputting actions directly constrains models to the training action space (typically planar differential drive), limiting generalization to platforms like aerial robots. Second, to learn a policy mapping from observations to actions, these methods require training examples where the robot traverses from the current view to the goal image. This restricts training to start and goal images from the same trajectory, and thus implicitly assumes that the deployment camera shares the characteristics of the recording device. We show that deviations in camera intrinsics or robot embodiment therefore lead to degraded performance. Third, relying on a single subgoal image makes the system sensitive to subgoal selection errors. If an incorrect target is chosen, the navigation policy may fail, as the robot cannot visually match its current view to the incorrect subgoal.

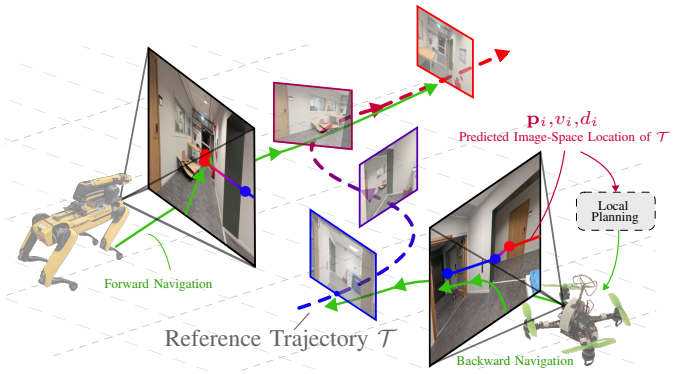

Fig. 1. Given only a reference trajectory of (unposed) RGB images $\mathcal{T}$, our model localizes the trajectory within the robot's current view. The predicted image-space coordinates, distances and visibility of the reference trajectory poses $(\mathbf{p}_i, d_i, v_i)$ provide robot-agnostic guidance for local planning, enabling different robots to go to any point on the trajectory, from any view of the trajectory.

We approach the problem of visual navigation by decoupling perception from action, and propose to learn a model that predicts the reference trajectory directly in image space, such that it can be tracked by conventional local planners.

To this end, we present LoTIS, a model for visual navigation that given a reference trajectory (sequence of RGB images) predicts the following representation: 1) the image-space coordinates of where each trajectory pose would appear in the robot's current view, 2) whether these poses are visible, and 3) a normalized distance to each visible pose, see Fig. 1. This overcomes the aforementioned challenges: First, our model predictions can be easily paired with downstream controllers or planners for diverse robot embodiments, without further training. Second, because our model learns to localize trajectory poses rather than imitate actions, we can construct training pairs by sampling reference and query images from different trajectories. As a result, we can explicitly train on data from mismatched cameras, varied mounting heights, and off-

This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation. The development was partly enabled by the Berzelius resource provided by the Knut and Alice Wallenberg Foundation at the National Supercomputer Centre.

The authors are with the Division of Robotics, Perception, and Learning, KTH Royal Institute of Technology, Sweden, and also affiliated with Digital Futures. Contact: {flbusch, vahs, quantao, jgop, yixica, tumova, olovand}@kth.se

trajectory viewpoints, enabling robustness to such variations. Finally, processing the full trajectory sequence rather than selecting a single subgoal image overcomes the sensitivity of accurate subgoal selection in existing works, leading to substantial performance gains in navigation success rate across all evaluations.

In addition, our model enables new capabilities, most notably backward traversal, where related approaches largely fail while our method maintains robust performance. This enables, for the first time, using an RGB reference trajectory in the general setting where the robot can navigate from any view of the trajectory to any point on it. This means a single phone camera recording is enough for different robots to navigate to any point along it, in either direction, without needing to re-record or recalibrate for a new platform.

In summary, we decouple perception from action and provide a learned perception model that interfaces with classical planners, with the following contributions:

1) We propose an image-space representation, defined by 2D coordinates, visibility, and distance of each reference trajectory pose in the robot's current view, that easily pairs with local planners to guide diverse robots in image space. In real-world indoor and outdoor experiments, we show that this representation is suitable to guide both a quadrotor and a quadruped from a single phone-recorded trajectory.
2) We introduce a cross-trajectory training strategy tailored for this representation, where reference and query images are sampled from different trajectories, exposing the model to camera mismatches and challenging viewpoints. This enables robust backward traversal and consequently supports navigation to any point on the trajectory.
3) We propose a model architecture that processes the full reference trajectory jointly rather than relying on single subgoal-selection, while enabling real-time deployment on embedded hardware. We show that for existing methods, reliance on a single subgoal-selection leads to decreased localization accuracy with distance from the trajectory, whereas our joint processing enables LoTIS paired with a local planner to maintain robust success rates even when initialized far from the trajectory.

A. Problem Statement

We consider the task of navigating to any point along a recorded RGB reference trajectory using only RGB images. Formally, given a reference trajectory $\mathcal{T} = \{I_1, \dots, I_N\}$ consisting of N RGB images, and a query image I_q from the robot's current viewpoint, the goal is to navigate to any point on the trajectory, indexed by $g \in \{1, \dots, N\}$, starting from any position in the trajectory's vicinity, as long as there is visual overlap with some portion of $\mathcal{T}$.

We make no explicit assumptions about camera calibration or robot embodiment. As such the reference trajectory $\mathcal{T}$ may be recorded with a different camera or platform than the one used for navigation.

II. Method

We approach this problem by decoupling perception from action and learn a model that provides robot-agnostic guidance that can be easily consumed by classical motion planning and control stacks designed for specific embodiments. To this end, we propose LoTIS, a model that predicts where the poses of a reference trajectory appear in the robot's current view in image-space. As this output is formulated in the robot's current view, it can easily be used by downstream controllers, e.g. by simply steering towards the predicted points.

A. Guidance Representation

Our model's output provides guidance to be used by classical motion planning stacks. Formally, in the image-space of the robot's current view, we predict a triplet $(\mathbf{p}_i, v_i, d_i)$ for each reference frame $I_i \in \mathcal{T}$:

- A 2D point in image-space $\mathbf{p}_i \in \mathbb{R}^2$: where the camera pose corresponding to trajectory image I_i would appear in the query view, normalized to $[-1, 1] \times [-1, 1]$
- A visibility logit $v_i \in \mathbb{R}$: whether the pose is visible or occluded/out of view
- A normalized distance $d_i \in [0, 1]$: relative distance to the pose from the current query viewpoint. For scale-independency, we normalize the distances such that the farthest visible point is at distance 1.

B. Model Architecture

We design our model to process the reference trajectory $\mathcal{T}$ and match this with the current view of the robot I_q. Unlike most existing methods [11, 13, 16, 15] that match the current view pairwise to each image on the trajectory, we propose processing the full trajectory jointly and matching the robot's view against that. To ensure real-time deployment, we propose an asymmetric model architecture: a large *trajectory encoder* $\mathcal{E}_\mathrm{T}$ runs once at deployment to process the full trajectory, while a lightweight *query encoder* $\mathcal{E}_\mathrm{q}$ and *decoder* run efficiently online to produce the final predictions. The decoder consists of query-trajectory fusion $\mathcal{F}_\mathrm{qT}$, which matches I_q against the processed trajectory to find visual correspondences, and aggregates those into a global context within one summary token $\mathbf{c}_i$ per frame i on the reference trajectory; and a prediction head predicts the one final prediction $\mathbf{p}_i, v_i, d_i$ per frame i from the summary tokens. An overview is provided in Fig. 2

We use DINOv3 [12] as a frozen backbone to extract initial features for all images, given the strong performance of the DINO family on geometric vision tasks [19].

1) Trajectory Encoder $\mathcal{E}_\mathrm{T}$: Given the reference trajectory $\mathcal{T} = \{I_1, \dots, I_N\}$, we extract per-frame features using frozen DINOv3, project to dimension D, and prepend a learnable *summary token* $\mathbf{c}_i$ per frame. The resulting tokens are processed by L transformer blocks, each alternating between global attention (across all frames and patches) and frame-wise attention (within each frame), following [19]. We incorporate temporal structure via rotary position encodings (RoPE [14]).

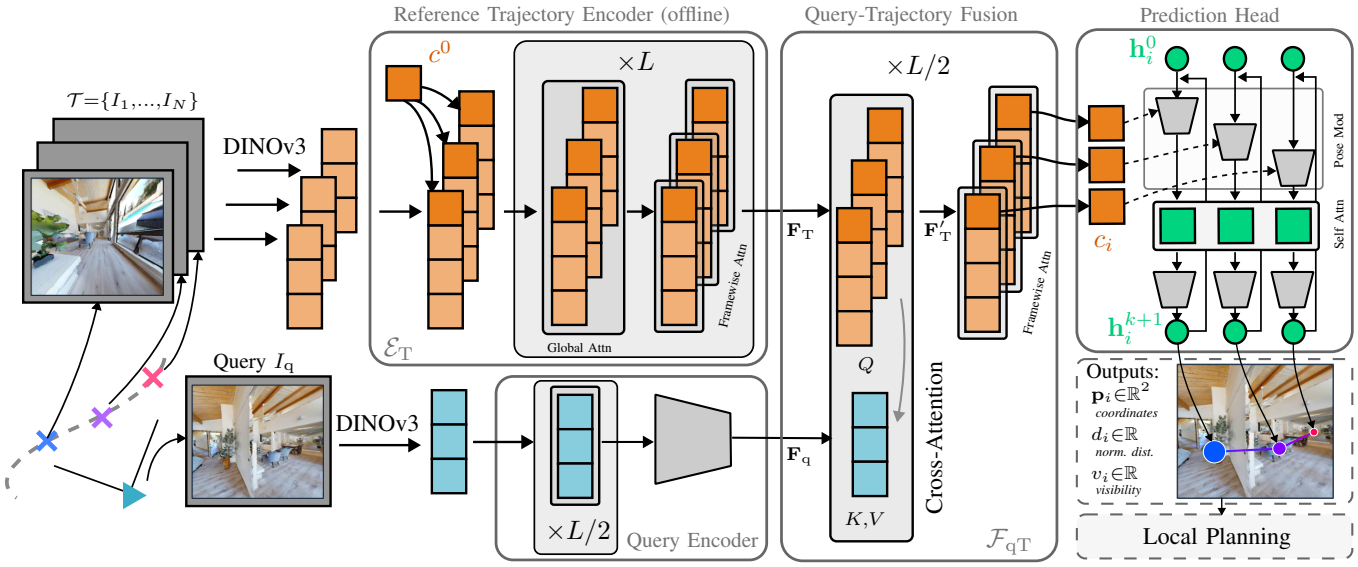

Fig. 2. **LoTIS Architecture.** Reference trajectory $\mathcal{T}$ and query I_q are processed by frozen DINOv3 backbones. A trajectory encoder ($\mathcal{E}_T$) captures spatio-temporal context once (offline), while a query encoder ($\mathcal{E}_q$) and query-trajectory fusion ($\mathcal{F}_{qT}$) perform online feature extraction and fusion, respectively. Finally, a recurrent transformer iteratively regresses image-space coordinates ($\mathbf{p}_i$), visibility (v_i), and distances (d_i).

The encoder outputs $\mathbf{F}_\mathrm{T} \in \mathbb{R}^{N\times(P+1)\times D}$, where P is the number of patches per image.

2) Query Encoder $\mathcal{E}_\mathrm{q}$: The query image I_q is processed by the same frozen DINOv3 backbone, projected to dimension D, and refined through $L/2$ self-attention layers with spatial RoPE, producing tokens $\mathbf{F}_\mathrm{q} \in \mathbb{R}^{P\times D}$. A linear adapter aligns features with the trajectory encoder's representation space.

3) Query-Trajectory Fusion $\mathcal{F}_\mathrm{qT}$: This module fuses query and trajectory features $\mathbf{F}_\mathrm{q}, \mathbf{F}_\mathrm{T}$ through $L/2$ blocks, each alternating cross-attention and frame-wise self-attention. For cross-attention, trajectory features $\mathbf{F}_\mathrm{T}$ (patches and summary tokens) serve as queries, while query image tokens $\mathbf{F}_\mathrm{q}$ serve as keys and values:

$$\mathbf{F}'_\mathrm{T} = \mathrm{CrossAttn}(Q{=}\mathbf{F}_\mathrm{T}, K{=}\mathbf{F}_\mathrm{q}, V{=}\mathbf{F}_\mathrm{q}). \tag{1}$$

This allows each trajectory frame to attend to the query view and find visual correspondences. The subsequent frame-wise self-attention aggregates these local correspondences into global context within each frame's summary token $\mathbf{c}_i$.

4) Prediction Head $\mathcal{P}$: We employ a recurrent transformer regressor, adapting the iterative refinement architecture from [19]. The head maintains a latent estimate $\mathbf{h}_i \in \mathbb{R}^D$ for each trajectory frame $i \in \{1,\ldots,N\}$, initialized by a learnable query. Over K iterations, the current estimate $\mathbf{h}^{(k)}$ is projected and used to condition the summary tokens $\mathbf{c}_i$ via Adaptive Layer Normalization (AdaLN [7]). The modulated tokens are processed by self-attention across all N frames, enabling the model to enforce geometric consistency along the trajectory. A linear layer predicts residual updates, and the estimate is refined as $\mathbf{h}^{(k+1)} \leftarrow \mathbf{h}^{(k)} + \Delta\mathbf{h}$.

After K iterations, the final estimates are projected to the output space: image coordinates $\mathbf{p}_i$ via $\tanh$ (normalized to $[-1,1]^2$), visibility logits v_i, and normalized distances d_i via scaled $\tanh$ (bounded to $[0,1]$).

5) Implementation: We use $L{=}12$ layers, hidden dimension $D{=}256$, and $K{=}4$ refinement iterations. This yields $\sim$50 M parameters, with $\sim$32 M in the trajectory encoder. Further details are provided in the extended paper version.

C. Training

1) Cross-Trajectory Training: We train on a combination of real-world navigation datasets and synthetic data generated in simulation [10].

A key advantage of our representation is that it enables *cross-trajectory* sampling: For real-world datasets, we sample the reference trajectory $\mathcal{T}$ and query image I_q from *different* trajectories within the same environment. We do this to encourage generalization, as this exposes the model to camera mismatches, off-trajectory viewpoints and environmental variations between traversals (lighting changes, dynamic objects, seasonal differences). Importantly, cross-trajectory sampling is only possible because we do not require action annotations between reference trajectory and query image.

In simulation, we generate reference trajectories between random start and goal points, sampling query images from random poses in the vicinity of these trajectories. To encourage generalization, we independently randomize camera parameters (field-of-view, aspect ratio, mounting height) for reference and query views, and apply rotational perturbations to query and trajectory poses before recording the images. Trajectory frames are sampled at stochastic intervals to vary sequence density for both simulation and real-world datasets.

Ground truth labels are generated via geometric projection using camera poses and depth maps. For real-world data, we preprocess depth maps with PriorDepthAnything [20].

2) Losses: We optimize the model using a weighted sum of three objectives:

$$\mathcal{L} = \lambda_\mathrm{pos}\mathcal{L}_\mathrm{pos} + \lambda_\mathrm{vis}\mathcal{L}_\mathrm{vis} + \lambda_\mathrm{dist}\mathcal{L}_\mathrm{dist}, \tag{2}$$

where $\mathcal{L}_{\text{pos}}$ is an L_1 loss applied to the predicted coordinates $\mathbf{p}_i$, masked to only penalize points where the ground truth is visible ($v_i^* = 1$). $\mathcal{L}_{\text{vis}}$ is a Binary Cross-Entropy loss applied to the visibility logits. $\mathcal{L}_{\text{dist}}$ is an L_1 loss applied to the normalized distance predictions. We compute this loss on the output of each iteration of the prediction head and apply temporal weighing [17].

3) Datasets: Our training data spans diverse environments, including simulation (HM3D [9], HSSD [2], AI2-THOR [3]) and real-world trajectories (CODa [23], LILocBench [18], BotanicGarden [4], TartanGround [6]). This mixture exposes the model to cluttered indoor spaces, unstructured outdoor paths, and urban settings with dynamic objects, as well as seasonal and day-night variations.

In total, we use approximately 25,000 reference trajectories (up to 40 frames each) and 850,000 query images across 500 unique environments. We train on a single NVIDIA RTX 5090 for approximately 4 days.

D. Navigation

Our model outputs a set of points $\mathbf{p}_i$ in the robot's current image frame representing the reference trajectory. To navigate, a downstream controller uses these predictions alongside a user-specified goal index $g \in \{1, \dots, N\}$ to determine the appropriate actions. We demonstrate this flexibility on two distinct controllers:

1) Yaw Controller with constant forward velocity: To demonstrate robust performance with minimal complexity, we employ a simple controller that controls only yaw velocity while commanding constant forward velocity. The controller identifies visible points $\mathcal{I}_{\text{vis}} = \{i \mid \sigma(v_i) > 0.5\}$, selects the closest one ($k = \text{argmin}_{i \in \mathcal{I}_{\text{vis}}} d_i$), and applies an offset in the desired direction of travel. We apply a proportional controller to derive yaw velocity commands that steer toward this target while commanding constant forward velocity.

2) Model Predictive Path Integral Control: To demonstrate that our model predictions can be easily combined with more sophisticated control strategies, we employ a perception-aware Model Predictive Path Integral (MPPI) controller [21] for a drone. We formulate a cost function that ensures that our predictions remain in the robot's FOV, aligning with similar approaches in perception-aware MPC [1, 5], and further incorporate proactive collision-avoidance. To this end, we use depth maps predicted by UniDepthV2 [8] to ground the model's predictions in 3D and to formulate collision avoidance. We refer the reader to the extended paper version for details on both controllers.

III. Simulation Experiments

We evaluate our method in photo-realistic simulation to assess robustness to environmental variations and to enable reproducible experiments.

A. Experimental Setup

We utilize the Gibson Habitat split [22] (5 environments) and HM3D [9] (100 environments) within the Habitat simulator, using 100 reference trajectories per dataset with two randomized initial poses per trajectory. All evaluation environments are held out from training; consequently, both the scenes and all reference trajectories used for evaluation are unseen during training. To test robustness against camera mismatch, we define two setups: *1) Matched Camera:* the query agent uses the exact same camera and mounting height as the reference, and *2) Cross-Camera:* the query agent has mismatched FOV (avg. $20°$, max $60°$), aspect ratio (avg. 0.5, max 1.5), and mounting height (avg. $0.5\,\text{m}$, max $1.2\,\text{m}$). We also apply rotational noise to reference trajectory poses to simulate imperfect recording.

We evaluate three tasks: **To End** (forward navigation to the final frame I_N, the standard task for all baselines); **To Start** (backward navigation to the first frame I_1, opposing the recorded views); and **Any Point** (navigation to a randomly selected frame I_k). For To End and To Start we test both **On-Trajectory** (agent starts on the reference path) and **Off-Trajectory** (agent starts at a random nearby pose with visual overlap) initialization; Any Point uses off-trajectory only.

Metrics. We report Success Rate (SR) and SPL; a run succeeds if the agent arrives within $0.5\,\text{m}$ of the goal within 1000 steps.

Baselines. We compare against ViNT [11], NoMaD [13], PlaceNav [15], and FAINT [16] using official pre-trained weights; all operate on a topological graph over the reference trajectory. Full baseline descriptions are provided in the extended paper.

We use the yaw controller with constant forward velocity, see Sec. II-D, paired with LoTIS for our simulation experiments. We also report LoTIS paired with simple reactive obstacle avoidance. Baselines integrate collision handling into their learned policies and cannot easily be augmented with external modules, illustrating a practical benefit of decoupling perception and control.

B. Results

We summarize our simulation results in Table I.

1) Forward On-Trajectory Navigation: We first evaluate the standard navigation task: following a reference trajectory forward from an on-trajectory start. LoTIS achieves a 94.7% success rate (SR) on Gibson and 98.5% on HM3D, outperforming the strongest baseline (ViNT) by 24.3 and 32.8 percentage points, respectively. Notably, when paired with simple obstacle avoidance, our method achieves 100% SR on both datasets. We hypothesize that this performance gap is largely due to how the reference trajectory is processed: while baselines extract a single subgoal, LoTIS localizes the entire sequence in image-space. This likely provides a much richer and more consistent guidance signal, which even a basic downstream controller (controlling only yaw angle) can exploit to achieve superior performance.

2) Off-Trajectory Initialization: A common failure mode for visual navigation is the inability to recover when the robot begins far from the reference trajectory. As shown in the results, baseline performance drops sharply in the "Off-Trajectory" setting. For example, ViNT's success rate on

TABLE I

NAVIGATION PERFORMANCE: WE REPORT SUCCESS RATE (SR) AND SPL (IN PARENTHESES). **CROSS**: QUERY CAMERA DIFFERS FROM REFERENCE CAMERA. **MATCHED**: SAME CAMERA PARAMETERS. **OFF-TRAJECTORY**: ROBOT INITIALIZED AWAY FROM THE TRAJECTORY. **ON-TRAJECTORY**: ROBOT INITIALIZED ON THE TRAJECTORY. FOR DETAILS, SEE SEC. III-A.

		To End (Forward)				**To Start (Backward)**				**Any Point**	
		On-Trajectory		Off-Trajectory		On-Trajectory		Off-Trajectory		Off-Trajectory	
	Method	Matched	Cross	Matched	Cross	Matched	Cross	Matched	Cross	Matched	Cross
Gibson	ViNT [11]	70.4 (67.8)	21.1 (17.8)	40.1 (30.7)	21.1 (17.5)	1.3 (0.9)	3.9 (3.4)	9.2 (8.1)	9.9 (7.8)	27.6 (22.8)	21.7 (19.3)
	PlaceNav [15]	53.9 (52.0)	5.3 (4.9)	15.1 (12.8)	11.2 (10.5)	3.9 (3.7)	4.6 (3.9)	9.2 (9.0)	10.5 (9.9)	22.4 (19.5)	13.8 (12.2)
	NoMaD [13]	23.0 (20.2)	8.6 (7.4)	24.3 (17.0)	17.1 (11.8)	8.6 (7.0)	7.2 (5.8)	11.2 (8.7)	11.2 (9.0)	31.6 (22.4)	27.0 (19.8)
	FAINT [16]	50.7 (47.8)	34.2 (31.0)	50.0 (42.5)	41.4 (33.6)	11.8 (9.3)	9.2 (7.5)	11.2 (10.1)	13.2 (10.4)	52.0 (42.6)	40.8 (35.2)
	LoTIS (Ours)	**94.7** (94.7)	**83.6** (83.1)	**88.2** (85.0)	**82.2** (79.5)	**88.2** (84.2)	**80.9** (78.1)	**77.6** (72.4)	**71.7** (67.7)	**85.5** (82.7)	**81.6** (77.8)
	+ Obstcl. Avoidance	**100.0** (99.9)	**98.0** (96.1)	**98.7** (93.9)	**94.7** (87.8)	**97.4** (89.9)	**89.5** (83.6)	**96.7** (87.6)	**90.8** (81.9)	**98.0** (92.4)	**95.4** (87.9)
HM3D	ViNT [11]	65.7 (64.4)	12.7 (12.1)	24.0 (20.8)	12.3 (10.3)	3.4 (3.1)	2.5 (2.3)	4.4 (3.7)	5.4 (5.0)	20.1 (17.9)	10.3 (9.3)
	PlaceNav [15]	55.4 (54.5)	7.8 (7.3)	11.3 (10.0)	6.4 (5.0)	3.4 (3.2)	2.5 (2.1)	4.9 (3.4)	4.9 (4.0)	13.2 (11.4)	8.3 (7.5)
	NoMaD [13]	25.0 (22.2)	9.3 (8.2)	14.2 (11.2)	12.7 (10.0)	4.4 (4.0)	4.4 (3.9)	10.3 (8.9)	8.3 (7.1)	23.5 (16.3)	14.7 (12.3)
	FAINT [16]	60.3 (59.0)	34.8 (31.6)	46.1 (40.6)	28.9 (25.3)	7.8 (7.1)	11.3 (10.3)	10.8 (9.6)	10.8 (9.3)	36.3 (32.4)	26.5 (23.5)
	LoTIS (Ours)	**98.5** (98.4)	**90.2** (90.1)	**74.0** (72.3)	**74.5** (73.0)	**81.9** (79.4)	**69.6** (67.6)	**69.6** (65.6)	**65.2** (61.6)	**71.1** (69.5)	**71.6** (70.4)
	+ Obstcl. Avoidance	**100.0** (99.8)	**97.5** (96.7)	**95.1** (89.6)	**92.2** (85.1)	**96.1** (90.7)	**84.8** (80.2)	**92.6** (83.2)	**86.8** (76.2)	**94.1** (90.0)	**94.1** (88.2)

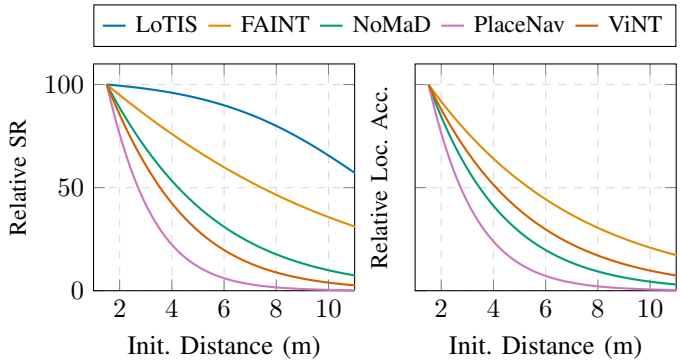

Fig. 3. Relative success rate (SR) for all methods on off-trajectory initialization over initialization distance (left), compared to subgoal localization accuracy for baseline methods (right). LoTIS does not perform discrete subgoal selection and is therefore omitted from the right panel.

HM3D falls from 65.7% to 24.0%.

Our model remains robust in these settings, maintaining 88.2% SR on Gibson. We observe lower off-trajectory performance on HM3D compared to Gibson (74.0% vs. 88.2%), which we attribute to HM3D's more cluttered environments. We note that the naive yaw controller at times causes the robot to get stuck behind small obstacles while attempting to return to the trajectory. By integrating basic obstacle avoidance, our success rate increases to 98.7% (Gibson) and 95.1% (HM3D).

As shown in Fig. 3, baseline subgoal localization accuracy drops rapidly with initialization distance, which results in SR degradation, whereas LoTIS maintains robust performance across all distances.

3) Camera Mismatch Robustness: The "Cross" columns in Table I evaluate each method's ability to handle mismatches in camera FOV, aspect ratio and camera mounting height. We observe a significant performance drop for all baselines here, e.g. ViNT's performance on Gibson drops from 70.4% to 21.1% in SR when the camera changes.

In contrast, LoTIS maintains high performance across all evaluations (e.g. 83.6% SR on Gibson, and 98.0% when paired with obstacle avoidance). We attribute this to our cross-trajectory training strategy, which likely helps increase robustness for the model against camera mismatch between query and reference trajectory. We note that our method is more sensitive to large height mismatches than FOV or AR mismatch, as this can lead to the trajectory being outside the field of view of the robot which may result in the robot getting lost. We provide a more detailed analysis of each method's sensitivity to different parameter mismatches in the extended paper version.

4) Backward Navigation: We evaluate the methods on the "To Start" task, where the robot must navigate to the first image of the reference trajectory. To successfully do this, the robot must be able to understand views that oppose the images of the reference trajectory. The results for this task are summarized in the "To Start" columns of Table I.

We observe a significant performance gap between LoTIS and the baseline methods in this setting. On the Gibson dataset (On-Trajectory/Matched),LoTIS achieves a 88.2% success rate (SR) or 97.4% when paired with obstacle avoidance. In contrast, the baselines largely fail: ViNT achieves 1.3%, NoMaD 8.6%, and FAINT 11.8%. This trend remains consistent across the HM3D dataset, where LoTIS maintains an 96.1% SR while all baselines remain below 8%. We note that here, even if subgoal selection is accurate, the baselines still largely fail due to the policy not being able to predict appropriate actions due to the robot's view opposing the chosen subgoal image.

When moving to the most challenging Off-Trajectory+Cross-Camera setting for backward navigation, LoTIS performance experiences a moderate decrease but remains mostly successful, with success rates between 65.2% and 86.8% (86.8% to 90.8% when paired with obstacle avoidance). Meanwhile, the success rates of the baseline methods stay within the 2% to 13% range.

LoTIS is not trained with explicit backward traversal demonstrations; this capability arises from avoiding direction-specific action priors, while cross-trajectory training exposes the model to opposing views of the same environment and joint trajectory processing resolves ambiguous frame matches.

TABLE II
REAL-WORLD NAVIGATION RESULTS. ALL TRIALS USE PHONE-RECORDED TRAJECTORIES WITH OFF-TRAJECTORY INITIALIZATION. INDOORS: CRAZYFLIE WITH MPPI. OUTDOORS: SPOT WITH YAW CONTROLLER.

	Indoors 1		Indoors 2		Outdoors 1		Outdoors 2	
	Fwd	Bwd	Fwd	Bwd	Fwd	Bwd	Fwd	Bwd
FAINT [16]	3/6	0/6	1/6	1/6	1/3	0/3	**3/3**	0/3
LoTIS (Ours)	**6/6**	**5/6**	**6/6**	**6/6**	**3/3**	**3/3**	**3/3**	**3/3**

5) Navigation to Arbitrary Goals (Any Point): The Any Point setting represents a more general usage of a reference trajectory where the robot is initialized off-trajectory and tasked to reach a certain point on the reference trajectory. This task serves as a summary for each method's performance as it requires strong capabilities in each of the aforementioned tasks. As shown in the rightmost columns of Table I, LoTIS maintains high performance in this setting, achieving an 85.5% SR on Gibson and 71.1% on HM3D (increasing to >94% with obstacle avoidance).

In summary, the results demonstrate that LoTIS paired with only a simple yaw controller achieves substantially higher success rates than the baselines across all evaluations, while enabling backwards traversal where baselines largely fail. When further paired with reactive collision avoidance, this leads to robust overall performance with over 95% success rates across most experiments.

IV. REAL-WORLD EXPERIMENTS

A. Evaluation Setup

We collect four previously unseen test-time reference trajectories using a handheld Google Pixel 6 smartphone: two indoors and two outdoors. We evaluate on a Crazyflie quadrotor with an analog FPV camera with MPPI control (Sec. II-D) for indoors trajectories, and on a Boston Dynamics Spot equipped with a Realsense D455 with our yaw controller (Sec. II-D), but leave its on-board collision avoidance enabled. All trials initialize off-trajectory. We compare against FAINT [16], the strongest baseline in the off-trajectory cross-camera simulation setting. Each trajectory is evaluated with 6 trials (indoors) or 3 trials (outdoors) per direction, totaling 36 runs per method. At deployment, each reference trajectory is uniformly subsampled to 40 frames, resized to 224×224, and encoded once before navigation; the cached trajectory features are then reused online. This offline step includes DINOv3 and the trajectory encoder. On the Jetson Orin AGX onboard Spot, offline encoding takes 120 ms and online inference 45 ms/frame, yielding ∼22 Hz. On the RTX 5090 used for Crazyflie, the corresponding times are 15 ms and 6 ms/frame. We provide videos of all real-world evaluations on the project page: https://finnbusch.com/lotis.

B. Results

As shown in Table II, our method achieves 100% success on forward navigation across all environments, while FAINT succeeds in only 27.8% of trials. This matches, or outperforms the results obtained in simulation. For indoors, we attribute the improved real-world performance compared to sim to the MPPI's ability to keep the predicted trajectory centered in view by actively matching the reference height. We note that for fairness, we manually adjust the drone's flight height to match the recording height for FAINT since FAINT solely provides actions in the horizontal plane. We show that the yaw controller is sufficient for the evaluated outdoors settings due to larger open spaces where Spot's internal reactive collision avoidance is sufficient, though it was not required to intervene during our evaluations. For both indoors and outdoors, we observe that FAINT being unable to reliably localize from off-trajectory initializations, is a main cause of failure.

The gap widens further on backward traversal: our method maintains 94.4% success (17/18), whereas FAINT largely fails (5.6%). These results match our simulation findings and suggest that our cross-trajectory training strategy and joint processing of the full trajectory enables the challenging task of backward traversal. Though performance remains consistent overall, we occasionally encounter views during backward traversal where our model can no longer match the view, and thus predicts no visible points. For the indoors setting, the MPPI is warm-started by the previous solution and will thus continue following the previous solutions, usually leading to better views and recovery within a few steps. For outdoors, we hypothesize that larger open spaces are less prone to produce views with no or little overlap with the trajectory.

V. CONCLUSION

We presented LoTIS, a model for visual navigation that predicts where a reference trajectory would appear in the robot's current view. Our approach provides zero-shot guidance for different embodiments, and enables backward traversal and robustness to camera mismatch—capabilities difficult for prior end-to-end methods. Experiments demonstrate 94-98% success on forward navigation across diverse embodiments both in simulation and real-world, and $5\times$ improvement on backward traversal, with real-world deployment confirming transfer to both quadrotor and quadruped platforms using phone-recorded trajectories. For the first time, we used a single RGB reference trajectory in the general setting where the robot can navigate from anywhere in the vicinity of, to any point on the trajectory, both forward and backward.

Limitations. Our method requires visual overlap between the current view and some portion of the reference trajectory; failures occur when this overlap is lost, particularly during backward traversal around sharp corners or under extreme viewpoint differences. Performance degrades with large camera height mismatches. Our real-world evaluation is limited to four environments and does not exhaustively cover all challenging scenarios where failure modes may differ. Longer trajectories require chunking to ensure coverage, which we demonstrate on the project page.

Future work. Promising directions include temporal memory for recovery when the trajectory leaves the field of view, height-robust guidance without direct visual overlap, and combining image-space guidance with semantic understanding to enable navigation to functional goals rather than trajectory indices.

REFERENCES

[1] Davide Falanga, Philipp Foehn, Peng Lu, and Davide Scaramuzza. Pampc: Perception-aware model predictive control for quadrotors. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1–8. IEEE, 2018.

[2] Mukul Khanna, Yongsen Mao, Hanxiao Jiang, Sanjay Haresh, Brennan Shacklett, Dhruv Batra, Alexander Clegg, Eric Undersander, Angel X Chang, and Manolis Savva. Habitat synthetic scenes dataset (hssd-200): An analysis of 3d scene scale and realism tradeoffs for objectgoal navigation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16384–16393, 2024.

[3] Eric Kolve, Roozbeh Mottaghi, Winson Han, Eli VanderBilt, Luca Weihs, Alvaro Herrasti, Matt Deitke, Kiana Ehsani, Daniel Gordon, Yuke Zhu, et al. Ai2-thor: An interactive 3d environment for visual ai. *arXiv preprint arXiv:1712.05474*, 2017.

[4] Yuanzhi Liu, Yujia Fu, Minghui Qin, Yufeng Xu, Baoxin Xu, Fengdong Chen, Bart Goossens, Poly ZH Sun, Hongwei Yu, Chun Liu, et al. Botanicgarden: A high-quality dataset for robot navigation in unstructured natural environments. *IEEE Robotics and Automation Letters*, 9(3):2798–2805, 2024.

[5] Ihab S Mohamed, Guillaume Allibert, and Philippe Martinet. Sampling-based mpc for constrained vision based control. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3753–3758. IEEE, 2021.

[6] Manthan Patel, Fan Yang, Yuheng Qiu, Cesar Cadena, Sebastian Scherer, Marco Hutter, and Wenshan Wang. Tartanground: A large-scale dataset for ground robot perception and navigation. *arXiv preprint arXiv:2505.10696*, 2025.

[7] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4195–4205, 2023.

[8] Luigi Piccinelli, Christos Sakaridis, Yung-Hsu Yang, Mattia Segu, Siyuan Li, Wim Abbeloos, and Luc Van Gool. Unidepthv2: Universal monocular metric depth estimation made simpler. *arXiv preprint arXiv:2502.20110*, 2025.

[9] Santhosh K Ramakrishnan, Aaron Gokaslan, Erik Wijmans, Oleksandr Maksymets, Alex Clegg, John Turner, Eric Undersander, Wojciech Galuba, Andrew Westbury, Angel X Chang, et al. Habitat-matterport 3d dataset (hm3d): 1000 large-scale 3d environments for embodied ai. *arXiv preprint arXiv:2109.08238*, 2021.

[10] Manolis Savva, Abhishek Kadian, Oleksandr Maksymets, Yili Zhao, Erik Wijmans, Bhavana Jain, Julian Straub, Jia Liu, Vladlen Koltun, Jitendra Malik, Devi Parikh, and Dhruv Batra. Habitat: A Platform for Embodied AI Research. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[11] Dhruv Shah, Ajay Sridhar, Nitish Dashora, Kyle Stachowicz, Kevin Black, Noriaki Hirose, and Sergey Levine. Vint: A foundation model for visual navigation. *arXiv preprint arXiv:2306.14846*, 2023.

[12] Oriane Siméoni, Huy V Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. Dinov3. *arXiv preprint arXiv:2508.10104*, 2025.

[13] Ajay Sridhar, Dhruv Shah, Catherine Glossop, and Sergey Levine. Nomad: Goal masked diffusion policies for navigation and exploration. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 63–70. IEEE, 2024.

[14] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[15] Lauri Suomela, Jussi Kalliola, Harry Edelman, and Joni-Kristian Kämäräinen. Placenav: Topological navigation through place recognition. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5205–5213. IEEE, 2024.

[16] Lauri Suomela, Sasanka Kuruppu Arachchige, German F. Torres, Harry Edelman, and Joni-Kristian Kämäräinen. Synthetic vs. real training data for visual navigation. In *arXiv preprint arXiv:2509.11791*, 2025.

[17] Zachary Teed and Jia Deng. Raft: Recurrent all-pairs field transforms for optical flow. In *European conference on computer vision*, pages 402–419. Springer, 2020.

[18] Niklas Trekel, Tiziano Guadagnino, Thomas Läbe, Louis Wiesmann, Perrine Aguiar, Jens Behley, and Cyrill Stachniss. Benchmark for evaluating long-term localization in indoor environments under substantial static and dynamic scene changes. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 10770–10777. IEEE, 2025.

[19] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. Vggt: Visual geometry grounded transformer. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 5294–5306, 2025.

[20] Zehan Wang, Siyu Chen, Lihe Yang, Jialei Wang, Ziang Zhang, Hengshuang Zhao, and Zhou Zhao. Depth anything with any prior. *arXiv preprint arXiv:2505.10565*, 2025.

[21] Grady Williams, Paul Drews, Brian Goldfain, James M Rehg, and Evangelos A Theodorou. Aggressive driving with model predictive path integral control. In *2016 IEEE international conference on robotics and automation (ICRA)*, pages 1433–1440. IEEE, 2016.

[22] Fei Xia, Amir R. Zamir, Zhiyang He, Alexander Sax, Jitendra Malik, and Silvio Savarese. Gibson Env: real-world perception for embodied agents. In *Computer Vision and Pattern Recognition (CVPR), 2018 IEEE Conference on*. IEEE, 2018.

[23] Arthur Zhang, Chaitanya Eranki, Christina Zhang, Ji-Hwan Park, Raymond Hong, Pranav Kalyani, Lochana Kalyanaraman, Arsh Gamare, Arnav Bagad, Maria Esteva, et al. Toward robust robot 3-d perception in urban environments: The ut campus object dataset. *IEEE Transactions on Robotics*, 40:3322–3340, 2024.